

Sous le thème
Rapport de projet fin d'année
3ème /4ème année
Ingénierie Informatique et Réseaux

Annee UNIVERSITAIRE : 2025-2026

TITRE DU THEME

Tastify

Application ERP de gestion de restaurant avec analyse de
sentiment des avis clients

Encadré par :

Tuteur de l'école :

Pr. Prénom et nom

Réalisé par :

Diouri Mehdi et El kherazi Ibtiha

Projet de Fin d'Année

Dédicaces

Nous dédions ce travail à nos familles, pour leur soutien constant, leur patience et leur confiance. Leur présence a permis de mener ce projet avec sérieux, même dans les phases où la charge technique et documentaire était importante.

Nous dédions également ce rapport à nos enseignants, qui nous ont transmis les bases nécessaires en génie logiciel, développement web, bases de données, modélisation, sécurité et conduite de projet. Enfin, nous adressons cette dédicace à toutes les personnes qui encouragent l'apprentissage par la pratique et l'amélioration continue.

Remerciements

Avant de présenter le projet Tastify, nous exprimons notre reconnaissance à nos familles pour leur accompagnement tout au long de notre formation. Nous remercions également l'ensemble du personnel enseignant et administratif pour le cadre académique qui a rendu possible la réalisation de ce Projet de Fin d'Année.

Nous remercions particulièrement les enseignants qui nous ont guidés dans les disciplines mobilisées par ce projet : conception UML, développement backend, développement frontend, architecture logicielle, bases de données, tests et documentation technique. Ces acquis constituent le socle sur lequel Tastify a été conçu et réalisé.

Résumé

Tastify est une application web full-stack destinée à centraliser la gestion quotidienne d'un restaurant. Le projet répond à des difficultés concrètes observées dans la restauration : dispersion des informations, erreurs de transmission entre la salle et la cuisine, suivi manuel des stocks, manque de visibilité sur les réservations, encaissements complexes et exploitation limitée des avis clients.

La solution repose sur un backend Django REST Framework, une base de données MySQL, Redis pour les échanges temps réel et les traitements asynchrones, Celery pour les tâches de fond, ainsi que deux applications React : une interface staff pour le gérant, le serveur et le cuisinier, et un portail client. Les modules confirmés par le dépôt couvrent le menu, les tables, les commandes, le Kitchen Display System, les stocks, les ressources humaines, les réservations, les paiements, la fidélité, les avis, les tableaux de bord et la configuration du restaurant.

Une partie importante du projet concerne l'analyse de sentiment des avis clients. Le code implémente un pipeline asynchrone qui analyse les commentaires au moyen de l'API Hugging Face lorsque le jeton est configuré. Il utilise le modèle multilingue NLP Town pour les avis non arabes, MARBERT pour les commentaires en arabe, et un secours local par mots-clés nommé `mots-cles-simple`. Les identifiants exacts des modèles sont détaillés dans le chapitre consacré à l'analyse de sentiment. Cette fonctionnalité aide le gérant à suivre la satisfaction client sans interrompre l'expérience utilisateur.

Mots-clés : ERP, restaurant, Django, React, MySQL, Redis, Celery, WebSocket, KDS, avis clients, analyse de sentiment, Hugging Face, PFA.

Abstract

Tastify is a full-stack web application designed to centralize daily restaurant management. The project addresses practical issues such as scattered operational information, communication errors between waiters and kitchen staff, manual inventory tracking, limited reservation visibility, complex payment flows and underused customer feedback.

The solution is based on a Django REST Framework backend, a MySQL database, Redis for real-time communication and asynchronous processing, Celery for background tasks, and two React applications : a staff interface for managers, waiters and kitchen staff, and a customer portal. The repository confirms modules for menu management, tables, orders, Kitchen Display System, inventory, human resources, reservations, payments, loyalty, reviews, dashboards and restaurant settings.

A key part of the project is customer review sentiment analysis. The implemented pipeline processes reviews asynchronously through the Hugging Face API when a token is configured. It uses the NLP Town multilingual model for non-Arabic reviews, MARBERT for Arabic comments, and a local keyword-based fallback named `mots-cles-simple`. The exact model identifiers are detailed in the sentiment analysis chapter. This feature helps managers monitor customer satisfaction without blocking the user experience.

Liste des abréviations

Abréviation	Signification
API	Application Programming Interface.
ASGI	Asynchronous Server Gateway Interface.
CRUD	Create, Read, Update, Delete.
DRF	Django REST Framework.
ERP	Enterprise Resource Planning, progiciel de gestion intégré.
JWT	JSON Web Token.
KDS	Kitchen Display System, écran de suivi des commandes en cuisine.
ML	Machine Learning, apprentissage automatique.
NLP	Natural Language Processing, traitement automatique du langage naturel.
ORM	Object-Relational Mapping.
PFA	Projet de Fin d'Année.
PWA	Progressive Web App.
RBAC	Role-Based Access Control, contrôle d'accès basé sur les rôles.
REST	Representational State Transfer.
SPA	Single Page Application.
SQL	Structured Query Language.
UML	Unified Modeling Language.
WS	WebSocket.

Informations à confirmer

La page de garde reprend le gabarit Word fourni. Les noms des étudiants et l'année universitaire ont été confirmés par l'utilisateur. Les champs restants qui ne sont pas vérifiables dans les fichiers fournis ne sont pas inventés.

- Le gabarit DOCX contient **Tuteur de l'école : Pr. Prénom et nom**. Le nom exact de l'encadrant n'est pas vérifiable dans les fichiers exploités.
- Les captures présentes dans le dépôt sont réutilisées uniquement lorsqu'elles correspondent à des modules effectivement observés. Les captures manquantes sont listées en annexe au lieu d'être fabriquées.

Table des matières

Dédicaces	i
Remerciements	ii
Résumé	iii
Abstract	iv
Liste des abréviations	v
Informations à confirmer	vi
Introduction générale	1
1 Contexte général	2
Introduction	2
1.1 Contexte métier	2
1.2 Problématique	2
1.3 Etude de l'existant	3
1.4 Objectifs du projet	4
1.5 Périmètre fonctionnel retenu	4
1.6 Planification	5
Conclusion	6
2 Analyse des besoins et conception	7
Introduction	7
2.1 Acteurs du système	7

2.2	Besoins fonctionnels	8
2.3	Besoins non fonctionnels	8
2.4	Diagrammes de cas d'utilisation	9
2.5	Diagramme de classes	11
2.6	Modèle logique de données	11
2.7	Règles métier principales	12
2.8	Architecture fonctionnelle cible	13
	Conclusion	13
3	Analyse de sentiment et choix du modèle	15
	Introduction	15
3.1	Rôle de l'analyse de sentiment dans Tastify	15
3.2	Données réellement utilisées	16
3.3	Pipeline technique	16
3.4	Modèle réellement utilisé	17
3.5	Conversion des sorties	17
3.6	Comparaison avec des alternatives	18
3.7	Justification du choix	18
3.8	Limites de la fonctionnalité	20
	Conclusion	20
4	Architecture technique et réalisation	21
	Introduction	21
4.1	Stack technique	21
4.2	Architecture globale	22
4.3	Organisation du backend	23
4.4	API REST et documentation	23
4.5	Authentification et sécurité	24
4.6	Temps réel et Kitchen Display System	24
4.7	Applications frontend	25
4.8	Paieement et fidélité	25

4.9	Stocks et prévision simple	26
4.10	Interfaces réalisées	26
4.11	Déploiement Docker Compose	29
	Conclusion	29
5	Tests, validation, limites et perspectives	30
	Introduction	30
5.1	Stratégie de tests	30
5.2	Tests backend	31
5.3	Tests frontend et end-to-end	32
5.4	Validation de l'analyse de sentiment	32
5.5	Tests de charge	32
5.6	Limites du projet	33
5.7	Perspectives	33
	Conclusion	34
	Conclusion générale	35
A	Annexes	36
A.1	Schémas existants intégrés	36
A.2	Schémas techniques créés dans le rapport	36
A.3	Captures existantes réutilisées	37
A.4	Captures à fournir avant dépôt final	37
A.5	Informations non vérifiables ou incertaines	38
A.6	Commandes utiles	38

Table des figures

2.1	Diagramme de cas d'utilisation du back-office gérant	9
2.2	Diagramme de cas d'utilisation des modules salle et cuisine	10
2.3	Diagramme de cas d'utilisation du portail client	10
2.4	Diagramme de classes de Tastify	11
2.5	Architecture fonctionnelle générale de Tastify	13
3.1	Pipeline technique de l'analyse de sentiment	16
4.1	Architecture technique de Tastify	22
4.2	Séquence simplifiée d'authentification JWT	24
4.3	Flux temps réel entre prise de commande et KDS	25
4.4	Tableau de bord gérant du back-office	27
4.5	Interface serveur : plan de salle	27
4.6	Kitchen Display System côté cuisine	28
4.7	Portail client : consultation du menu	28
4.8	Organisation du déploiement Docker Compose	29

Liste des tableaux

1.1	Comparaison synthétique des solutions existantes	3
1.2	Périmètre fonctionnel confirmé	5
1.3	Planification pédagogique du projet	6
2.1	Acteurs et responsabilités	7
2.2	Besoins fonctionnels principaux	8
2.3	Besoins non fonctionnels	9
2.4	Synthèse du modèle logique de données	12
3.1	Données utilisées par le pipeline de sentiment	16
3.2	Normalisation des sorties de sentiment	18
3.3	Comparaison des approches de sentiment	19
4.1	Stack technique confirmée	22
4.2	Modules backend réalisés	23
4.3	Applications frontend	25
5.1	Familles de tests présentes dans le dépôt	30
5.2	Exemples de scénarios backend critiques	31
5.3	Parcours frontend à valider	32
A.1	Schémas existants intégrés	36
A.2	Schémas techniques créés	37
A.3	Captures existantes réutilisées	37
A.4	Captures à fournir ou remplacer avant dépôt final	38

Introduction générale

La transformation numérique des restaurants ne se limite plus à la présence en ligne ou à la prise de réservation. Elle touche désormais l'organisation opérationnelle : coordination entre la salle et la cuisine, disponibilité des plats, suivi des stocks, gestion des paiements, fidélisation et analyse des retours clients. Dans un établissement où plusieurs acteurs travaillent en parallèle, une information mal transmise peut provoquer un retard, une erreur de préparation, une rupture non anticipée ou une expérience client dégradée.

Le projet Tastify s'inscrit dans ce contexte. Il vise à proposer une application web centralisée pour la gestion d'un restaurant, en couvrant à la fois les besoins internes du personnel et les interactions côté client. Le dépôt montre une architecture full-stack composée d'un backend Django REST Framework, de services Redis et Celery, d'une base MySQL, de WebSocket via Django Channels et de deux applications React : une application staff et un portail client.

La particularité du projet réside dans son périmètre transversal. Tastify ne se limite pas à un module de menu ou à une simple caisse : il relie les tables, les commandes, le KDS, les stocks, les réservations, les paiements, la fidélité, les avis clients et les tableaux de bord. Cette intégration rend le projet pertinent pour un rapport de Projet de Fin d'Année, car elle mobilise plusieurs dimensions du génie logiciel : modélisation, architecture, API, sécurité, temps réel, asynchronisme, expérience utilisateur et validation.

Le rapport suit la structure de l'ancien rapport fourni, sans réutiliser son domaine métier. Le premier chapitre présente le contexte général, la problématique, l'étude de l'existant et les objectifs. Le deuxième chapitre formalise l'analyse des besoins et la conception. Un chapitre spécifique est consacré à l'analyse de sentiment, car cette fonctionnalité combine un enjeu métier et un pipeline technique précis. Les chapitres suivants décrivent l'architecture, la réalisation, les interfaces, les tests, les limites et les perspectives.

Un principe de rigueur guide la rédaction : les fonctionnalités sont décrites uniquement lorsqu'elles sont confirmées par le dépôt ou par les fichiers fournis. Les informations incertaines sont signalées explicitement, afin d'éviter de présenter au jury des éléments non vérifiables.

Chapitre 1

Contexte général

Introduction

Ce chapitre situe Tastify dans son environnement métier. Il présente les difficultés rencontrées dans la gestion d'un restaurant, formule la problématique, analyse brièvement les solutions existantes et définit les objectifs retenus pour le projet.

1.1 Contexte métier

Un restaurant fonctionne autour de flux courts et interdépendants. Une table se libère, une commande est prise, des plats sont envoyés en cuisine, des ingrédients sont consommés, un paiement est enregistré, puis le client peut laisser un avis. Ces actions mobilisent des rôles différents : le gérant, le serveur, le cuisinier et le client.

Lorsque ces informations sont gérées avec des outils séparés, le risque d'incohérence augmente. Le serveur peut ne pas connaître la disponibilité réelle d'un plat, la cuisine peut recevoir une commande incomplète, le gérant peut découvrir une rupture de stock trop tard et les avis clients peuvent rester dispersés. La numérisation d'un restaurant devient donc utile lorsqu'elle centralise les données et réduit les frictions entre les acteurs.

1.2 Problématique

La problématique principale du projet peut être formulée ainsi :

Comment concevoir une application web centralisée, maintenable et accessible permettant de piloter les opérations d'un restaurant, tout en améliorant la coordination entre la salle, la cuisine, la gestion administrative et l'expérience

client ?

Cette problématique se décline en plusieurs questions opérationnelles :

- comment séparer les accès selon les rôles sans multiplier inutilement les applications ;
- comment transmettre rapidement les commandes vers la cuisine ;
- comment relier commandes, stocks, paiements et fidélité ;
- comment exploiter les avis clients sans bloquer l’expérience utilisateur ;
- comment rendre l’ensemble testable et déployable avec une architecture claire.

1.3 Etude de l’existant

Le marché propose plusieurs solutions de gestion de restaurant ou de caisse connectée. Elles couvrent souvent une partie du besoin : prise de commande, caisse, stocks ou reporting. Le tableau 1.1 synthétise leur positionnement face aux besoins de Tastify.

TABLE 1.1 – Comparaison synthétique des solutions existantes

Solution	Points forts	Limites fréquentes	Lecture pour Tastify
Lightspeed Restaurant	Caisse, menu, commandes, reporting.	Coût récurrent, dépendance à un écosystème commercial.	Confirme l’intérêt d’une solution intégrée.
Toast POS	Orientation restaurant, commande et paiement.	Disponibilité et adaptation au marché local variables.	Met en évidence l’importance du lien salle-cuisine.
Revel Systems	Couverture ERP large, gestion multi-sites.	Complexité et coût élevés pour une petite structure.	Montre l’intérêt d’un périmètre modulaire.
L’Addition	Solution spécialisée restauration, caisse et service.	Couverture variable selon les offres et les intégrations.	Sert de référence pour les besoins opérationnels.
Tastify	Projet web modulaire, deux portails, temps réel, avis et fidélité.	Prototype académique, données de production non disponibles.	Solution adaptée au périmètre pédagogique du PFA.

L'étude de l'existant montre que les solutions commerciales sont puissantes, mais parfois coûteuses ou dépendantes d'un environnement matériel et contractuel. Tastify se positionne comme un projet académique complet, focalisé sur la compréhension des flux et sur une architecture réutilisable.

1.4 Objectifs du projet

Les objectifs du projet sont les suivants :

- centraliser la gestion des menus, catégories, plats, tables, commandes et réservations ;
- offrir une interface staff adaptée aux rôles gérant, serveur et cuisinier ;
- offrir un portail client pour consulter le menu, réserver, payer et suivre la fidélité ;
- améliorer la coordination cuisine grâce à un Kitchen Display System et aux Web-Socket ;
- gérer les stocks, les ingrédients, les recettes et les mouvements associés ;
- exploiter les avis clients grâce à une analyse de sentiment asynchrone ;
- produire une architecture testable, documentée et déployable avec Docker Compose.

1.5 Périmètre fonctionnel retenu

Le périmètre confirmé par le dépôt couvre les modules listés dans le tableau 1.2.

TABLE 1.2 – Périmètre fonctionnel confirmé

Domaine	Fonctions couvertes
Menu	Catégories, plats, images, disponibilité et liens avec les ingrédients.
Salle	Tables, plan de salle, statut de table et prise de commande.
Cuisine	KDS, lignes de commande, statuts de préparation et synchronisation temps réel.
Stock	Ingrédients, seuils d’alerte, recettes et mouvements de stock.
RH	Employés, shifts, offres d’emploi et candidatures.
Réservations	Création, disponibilité, conflits de créneaux et gestion côté client/staff.
Paielements	Paielement d’une commande, contribution par ligne et lien avec le client.
Fidélité	Profil, points, transactions et récompenses.
Avis	Commentaires, note optionnelle et analyse de sentiment.
Analytics	Indicateurs de tableau de bord et statistiques de sentiment.
Configuration	Informations du restaurant, devise, couleurs, paramètres de service.

1.6 Planification

Le dépôt ne contient pas de journal de projet complet permettant de prouver une planification datée. Le tableau 1.3 présente donc une planification pédagogique cohérente avec la structure du projet, à considérer comme une reconstitution documentaire et non comme un relevé historique certifié.

TABLE 1.3 – Planification pédagogique du projet

Phase	Contenu	Durée indicative
Phase 1	Initialisation du dépôt, environnement Docker, modèles principaux, authentification et rôles.	2 semaines
Phase 2	Modules gérant : menu, catégories, stocks, ressources humaines et configuration.	2 semaines
Phase 3	Salle et cuisine : tables, commandes, KDS, statuts et WebSocket.	2 semaines
Phase 4	Portail client : menu public, réservation, compte, panier et paiement.	2 semaines
Phase 5	Fidélité, avis, analyse de sentiment et indicateurs analytics.	2 semaines
Phase 6	Tests, documentation, conteneurisation et préparation du rapport.	1 semaine

Conclusion

Le contexte étudié justifie la création d’une solution centralisée et modulaire pour les restaurants. Tastify répond à cette problématique par une architecture full-stack couvrant les flux essentiels. Le chapitre suivant formalise les besoins, les acteurs et la conception du système.

Chapitre 2

Analyse des besoins et conception

Introduction

Ce chapitre formalise les besoins de Tastify et présente la conception fonctionnelle et objet. La modélisation s'appuie sur UML, langage standard de représentation des systèmes logiciels [3]. Les diagrammes intégrés proviennent des fichiers déjà présents dans le dépôt.

2.1 Acteurs du système

Tastify distingue quatre acteurs fonctionnels. Le système d'accès repose sur le rôle associé à l'utilisateur.

TABLE 2.1 – Acteurs et responsabilités

Acteur	Responsabilités principales
Gérant	Piloter le restaurant, gérer le menu, les stocks, les employés, les paramètres, les réservations, les avis et les indicateurs.
Serveur	Consulter le plan de salle, créer les commandes, suivre les tables et déclencher l'encaissement.
Cuisinier	Consulter les commandes cuisine, mettre à jour les statuts de préparation et signaler les plats prêts.
Client	Consulter le menu, s'inscrire, se connecter, réserver, payer, consulter sa fidélité et laisser un avis.

2.2 Besoins fonctionnels

Les besoins fonctionnels sont regroupés par domaine métier dans le tableau 2.2.

TABLE 2.2: Besoins fonctionnels principaux

Module	Besoins
Authentification	Inscription et connexion, séparation des portails staff/client, jetons JWT, refresh par cookie, contrôle des accès par rôle.
Menu	Créer, modifier, masquer ou afficher des catégories et des plats ; gérer la disponibilité et les images.
Salle	Visualiser les tables, leur capacité et leur statut ; accéder à la prise de commande par table.
Commandes	Créer des commandes, enregistrer les lignes, figer les prix unitaires et suivre les statuts de préparation.
Cuisine	Afficher les commandes dans un KDS et synchroniser les changements de statut vers le staff.
Stock	Gérer les ingrédients, les seuils d'alerte, les recettes et les mouvements de stock.
Réservations	Permettre au client de réserver et au staff de suivre les demandes selon la disponibilité.
Paiements	Enregistrer les paiements, gérer les méthodes disponibles et associer des contributions aux lignes de commande.
Fidélité	Suivre les points, transactions, paliers et récompenses clients.
Avis	Collecter les commentaires, stocker une note optionnelle et déclencher l'analyse de sentiment.
Analytics	Agréger des indicateurs : revenus, commandes, plats populaires, stocks et sentiment client.
Configuration	Paramétrer les informations du restaurant, la devise, certains réglages de service et l'identité visuelle.

2.3 Besoins non fonctionnels

Au-delà des fonctionnalités, Tastify doit satisfaire des contraintes techniques nécessaires à un projet maintenable.

TABLE 2.3 – Besoins non fonctionnels

Besoin	Traduction dans le projet
Sécurité	Authentification JWT, permissions DRF, rôles gérant/serveur/cuisinier/client, séparation des portails.
Maintenabilité	Découpage en applications Django spécialisées et deux frontends React séparés.
Réactivité	WebSocket pour les événements staff et KDS ; tâches Celery pour les traitements non bloquants.
Traçabilité	Modèles avec dates de création et de mise à jour ; conservation des prix de commande.
Déploiement	Docker Compose avec services backend, base de données, Redis, worker, beat et frontends.
Testabilité	Tests backend avec pytest, tests frontend avec Vitest et scénarios end-to-end avec Playwright.

2.4 Diagrammes de cas d'utilisation

Les diagrammes de cas d'utilisation décrivent les interactions principales entre les acteurs et les modules.

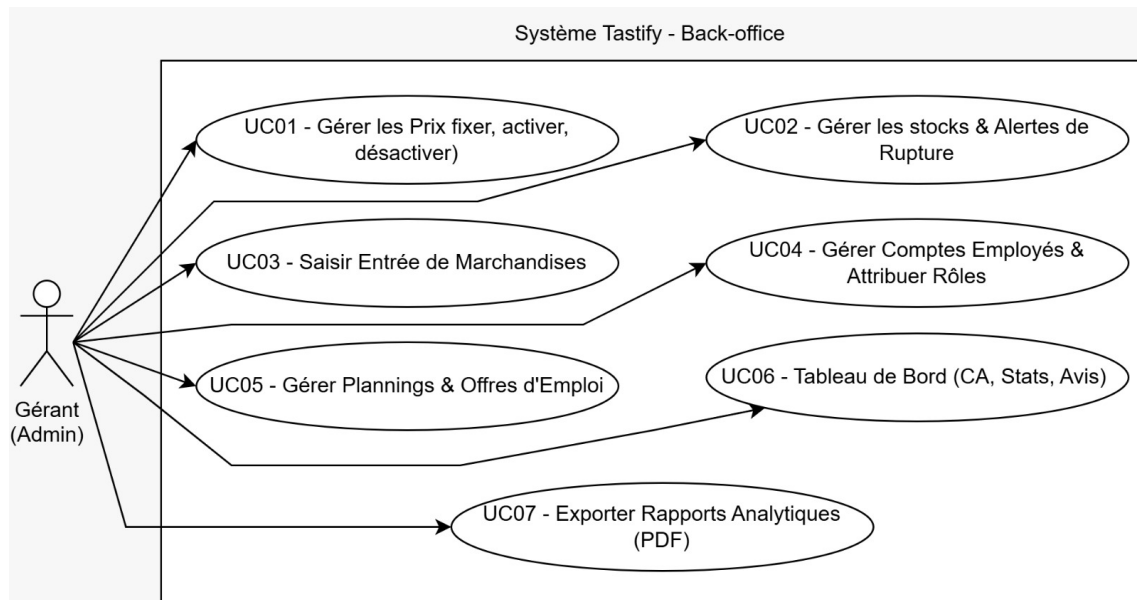


FIGURE 2.1 – Diagramme de cas d'utilisation du back-office gérant

Le diagramme 2.1 regroupe les fonctions d'administration : menu, stock, employés, alertes, tableaux de bord et rapports.

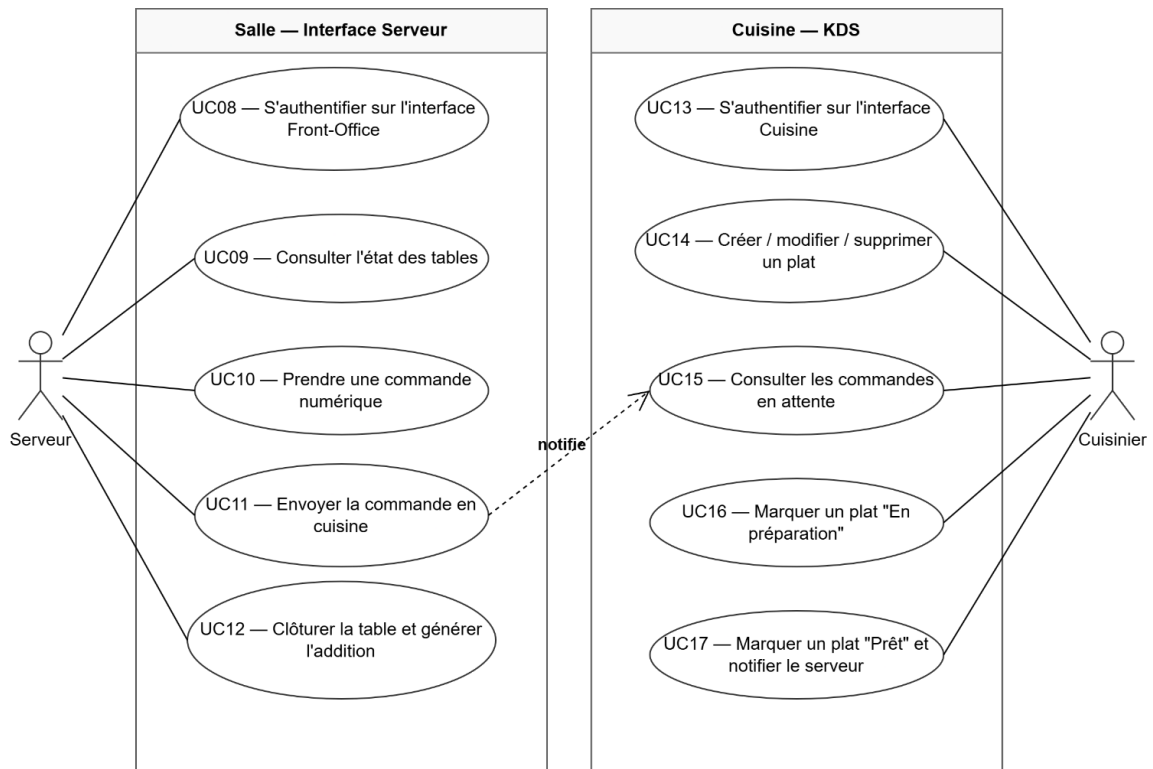


FIGURE 2.2 – Diagramme de cas d'utilisation des modules salle et cuisine

Le diagramme 2.2 illustre le lien entre le serveur, la salle et le KDS. Il justifie l'utilisation du temps réel, car la commande doit passer rapidement de l'interface serveur vers l'écran cuisine.

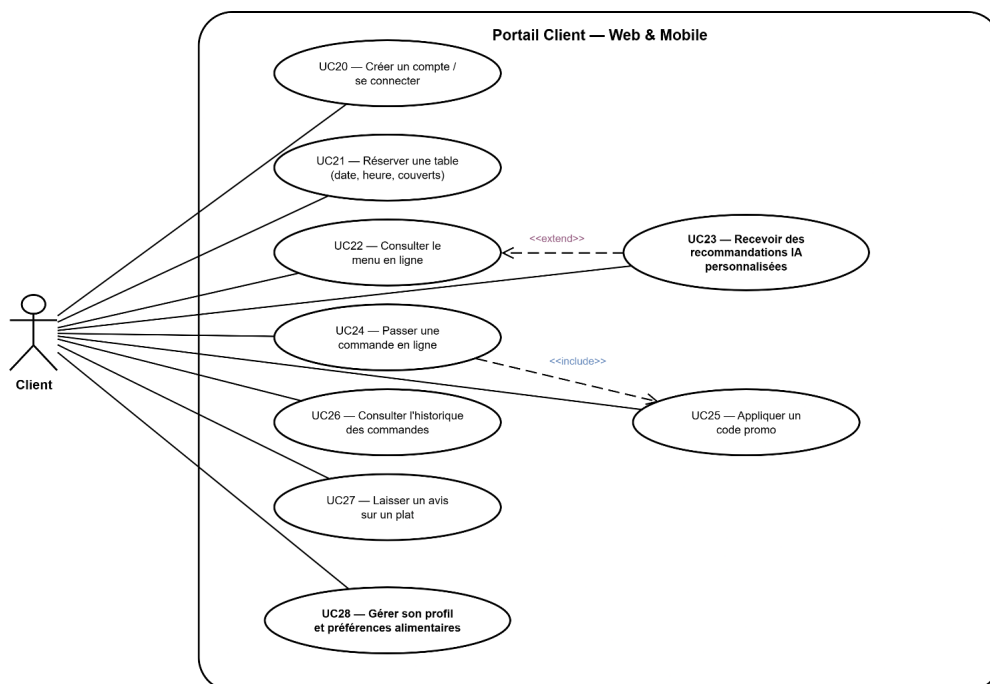


FIGURE 2.3 – Diagramme de cas d'utilisation du portail client

Le diagramme 2.3 représente les fonctions disponibles côté client : consultation du menu, réservation, commande, paiement, fidélité et avis.

2.5 Diagramme de classes

Le diagramme de classes présente les entités majeures et leurs relations.

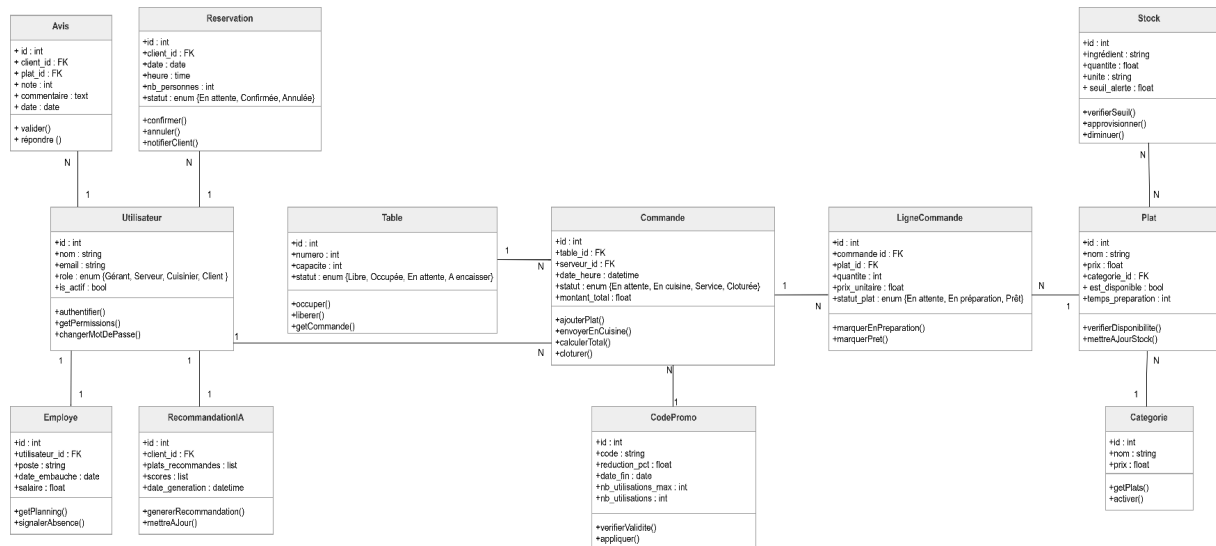


FIGURE 2.4 – Diagramme de classes de Tastify

Le diagramme 2.4 correspond aux domaines effectivement présents dans le backend Django : utilisateurs, plats, tables, commandes, réservations, paiements, avis et stock. Certaines entités détaillées dans le code ne sont pas toutes visibles sur le schéma initial ; elles sont donc précisées dans le modèle logique ci-dessous.

2.6 Modèle logique de données

Le modèle logique de données découle des modèles Django confirmés dans le dépôt. Le tableau 2.4 ne liste pas toutes les colonnes techniques, mais il synthétise les tables structurantes et leurs relations.

TABLE 2.4: Synthèse du modèle logique de données

Table / entité	Relations principales	Rôle
Utilisateur	Rôle applicatif.	Compte de connexion et séparation gérant, serveur, cuisinier, client.
Categorie, Plat	Catégorie 1..N plats.	Structure de la carte, prix, disponibilité, image et temps de préparation.
Ingredient, PlatIngredient	Relation N..M entre plats et ingrédients.	Recettes et quantités requises pour relier menu et stock.
Table, PlanText	Table liée aux commandes et réservations.	Plan de salle, capacité, position et statut opérationnel.
Commande, CommandeLigne	Commande 1..N lignes ; table, serveur ou client optionnels.	Prise de commande, suivi cuisine, prix figé et total.
Reservation	Client, table, date et créneau.	Réservation avec statut et contrôle de disponibilité.
Paielement, PaiementItem	Paielement lié à une commande ; contributions liées aux lignes.	Encaissement et partage possible de l'addition.
Avis, Analyse Sentiment	Avis 1..1 analyse ; avis lié à plat ou commande.	Commentaire client, note optionnelle, label et score de sentiment.
LoyaltyProfile, Reward	Profil 1..1 utilisateur ; transactions et récompenses.	Programme de fidélité.
WeatherData	Données datées.	Support aux indicateurs et prévisions simples.
Restaurant Configuration	Configuration unique.	Paramètres du restaurant, identité, devise et réglages de service.

2.7 Règles métier principales

Les règles suivantes structurent le comportement attendu :

- un utilisateur possède un rôle qui conditionne les routes et les actions accessibles ;
- une commande peut être liée à une table, à un serveur et éventuellement à un client ;
- chaque ligne de commande conserve le prix unitaire du plat au moment de la commande ;
- les statuts de lignes permettent de piloter le KDS : attente, préparation, prêt, servi ou annulé ;
- un paiement est toujours lié à une commande et peut comporter des contributions par ligne ;
- un avis appartient à un utilisateur et peut être lié à un plat ou à une commande ;
- une analyse de sentiment est associée à un seul avis ;
- le stock repose sur les ingrédients, les seuils d’alerte et les mouvements.

2.8 Architecture fonctionnelle cible

La conception distingue deux expériences applicatives : une application staff pour les rôles internes et un portail client. Les deux consomment le même backend API.

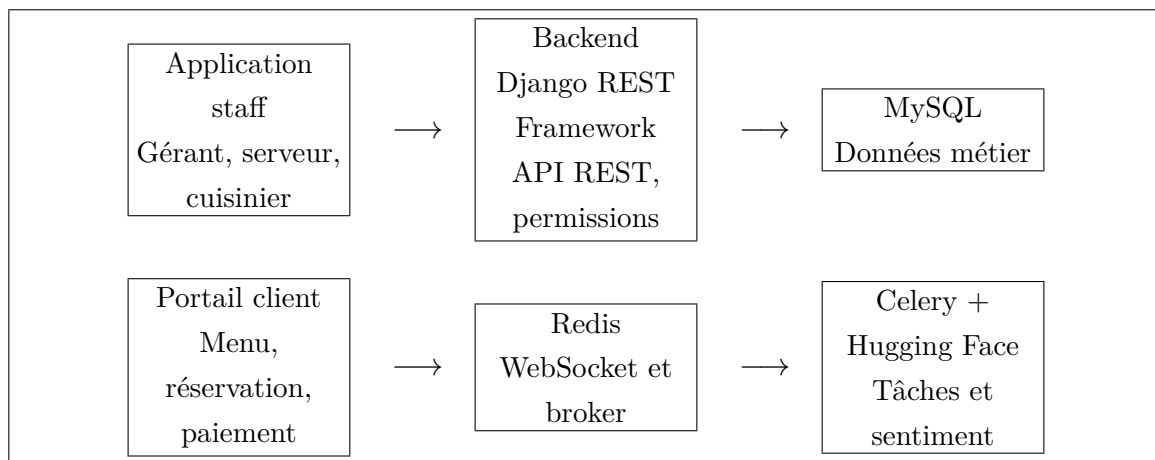


FIGURE 2.5 – Architecture fonctionnelle générale de Tastify

La figure 2.5 montre le choix central du projet : un backend unique expose les services communs, tandis que les interfaces sont séparées selon les usages.

Conclusion

L’analyse des besoins et la conception mettent en évidence un système multi-acteurs, organisé autour d’un backend commun et de deux interfaces web. Les diagrammes existants fournissent une base UML, complétée par le modèle logique issu du code. Le chapitre

suivant approfondit la fonctionnalité d'analyse de sentiment, essentielle pour exploiter les retours clients.

Chapitre 3

Analyse de sentiment et choix du modèle

Introduction

L'analyse de sentiment est la fonctionnalité d'intelligence applicative la plus explicite du projet. Elle ne remplace pas l'analyse humaine d'un gérant, mais elle transforme les avis textuels en indicateurs exploitables : polarité positive, neutre ou négative, score de confiance et modèle utilisé.

3.1 Rôle de l'analyse de sentiment dans Tastify

Dans Tastify, les avis clients constituent une source de retour direct sur l'expérience au restaurant. Un commentaire peut signaler un plat apprécié, un service lent, une mauvaise température, une attente trop longue ou une satisfaction globale. Sans traitement automatique, ces informations restent difficiles à suivre lorsque le volume augmente.

Le rôle de l'analyse de sentiment est donc double :

- **opérationnel**, car elle aide le gérant à repérer rapidement les tendances défavorables ;
- **analytique**, car elle alimente les statistiques de satisfaction affichées dans le tableau de bord.

Le dépôt confirme que les statistiques de sentiment sont agrégées dans le module **analytics**, à partir du modèle **AnalyseSentiment**. Le champ `Avis.sentiment_score` vaut +15 pour un avis positif, 0 pour un avis neutre et -15 pour un avis négatif.

3.2 Données réellement utilisées

Le projet ne contient pas de jeu de données local entraînant un modèle propriétaire. Les données utilisées par Tastify sont les avis saisis par les utilisateurs de l'application.

TABLE 3.1 – Données utilisées par le pipeline de sentiment

Champ	Utilisation
<code>Avis.commentaire</code>	Texte libre analysé par le pipeline. C'est l'entrée principale du modèle.
<code>Avis.note</code>	Note optionnelle entre 1 et 5. Elle est stockée mais le code d'analyse actuel se base sur le commentaire.
<code>Avis.lang_code</code>	Langue détectée simplement : ar si le texte contient des caractères arabes, en sinon.
<code>AnalyseSentiment.label</code>	Résultat normalisé : POSITIF , NEUTRE ou NEGATIF .
<code>AnalyseSentiment.score_brut</code>	Score de confiance renvoyé par Hugging Face ou par le fallback local.
<code>AnalyseSentiment.modele_utilise</code>	Nom du modèle distant ou de la méthode locale réellement utilisée.

Le texte transmis à Hugging Face est tronqué à 512 caractères dans la tâche Celery, ce qui correspond à une prudence technique adaptée aux avis courts et aux limites usuelles des modèles de classification.

3.3 Pipeline technique

Le pipeline est déclenché à la création d'un avis dans le `AvisViewSet`. La création de l'avis reste rapide, car l'analyse est déléguée à une tâche Celery.

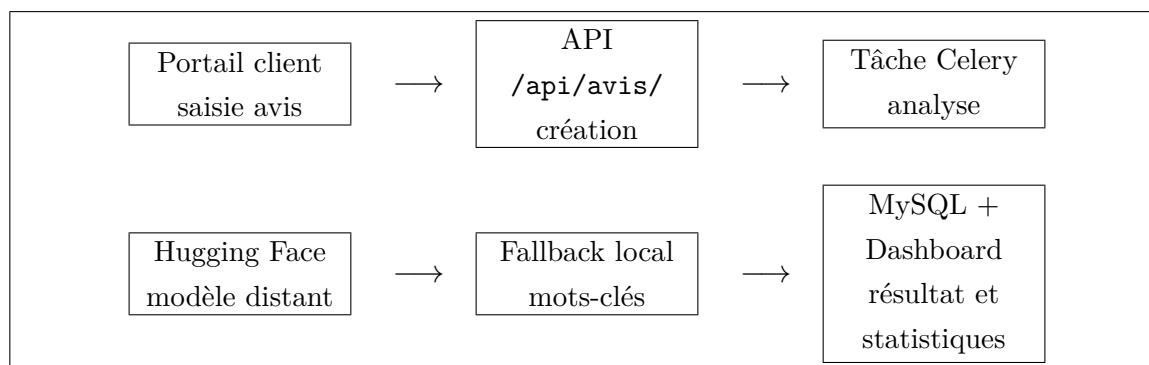


FIGURE 3.1 – Pipeline technique de l'analyse de sentiment

La figure 3.1 résume le flux : l'utilisateur soumet un avis, le backend l'enregistre, Celery effectue l'analyse, puis le résultat est persisté et agrégé.

3.4 Modèle réellement utilisé

Le code du fichier `apps/avis/tasks.py` définit explicitement les modèles suivants :

- `nlptown/bert-base-multilingual-uncased-sentiment` pour les avis non arabes ;
- `moussaKam/MARBERT-sentiment` pour les textes contenant des caractères arabes ;
- `nlptown/bert-base-multilingual-uncased-sentiment` comme modèle de secours si le modèle arabe ne répond pas ;
- `mots-cles-simple` comme secours local lorsque l'API Hugging Face n'est pas disponible ou lorsqu'aucun jeton n'est configuré.

Le modèle principal `nlptown/bert-base-multilingual-uncased-sentiment` est un BERT multilingue affiné pour l'analyse de sentiment sur des avis produits en six langues, dont le français [16]. Il renvoie une prédiction sous forme d'étoiles de 1 à 5, que le code convertit ensuite en trois classes : négatif, neutre ou positif. Ce choix est cohérent avec un projet de restaurant, car les avis clients ressemblent à des avis courts de produits ou services.

La base scientifique de ce type de modèle vient de BERT, qui apprend des représentations contextuelles bidirectionnelles et peut être adapté à des tâches de classification avec une couche de sortie [7]. Pour les textes arabes, le code s'appuie sur MARBERT, famille de modèles spécialisés dans l'arabe et ses variantes dialectales [1].

3.5 Conversion des sorties

Les sorties du modèle ne sont pas exposées directement au reste de l'application. Elles sont normalisées afin de simplifier l'exploitation métier.

TABLE 3.2 – Normalisation des sorties de sentiment

Sortie modèle	Label Tastify	Score métier
5 stars, stars, POSITIVE, LABEL_2	4 POSITIF	+15, avis favorable.
3 stars, NEUTRAL, LABEL_1	NEUTRE	0, signal sans polarité forte.
1 star, 2 stars, NEGATIVE, LABEL_0	NEGATIF	−15, avis défavorable à surveiller.

Cette conversion rend l’indicateur lisible pour le tableau de bord, tout en conservant dans `score_brut` la confiance renvoyée par le modèle.

3.6 Comparaison avec des alternatives

Le tableau 3.3 compare le modèle principal avec plusieurs alternatives pertinentes.

3.7 Justification du choix

Le choix du modèle `nlptown/bert-base-multilingual-uncased-sentiment` est défendable pour quatre raisons :

1. **Adéquation au domaine des avis.** Le modèle est affiné sur des avis, ce qui se rapproche des commentaires laissés sur des plats ou un restaurant.
2. **Support du français.** La fiche du modèle indique un entraînement incluant le français, ce qui correspond au contexte du rapport et aux interfaces francophones.
3. **Intégration rapide.** L’API Hugging Face permet de déléguer l’inférence sans embarquer un modèle lourd dans le backend.
4. **Robustesse opérationnelle.** Le fallback local permet de continuer à produire un résultat même sans jeton Hugging Face ou en cas d’erreur réseau.

Ce choix reste toutefois perfectible. Pour une version de production, il faudrait mesurer la qualité du modèle sur des avis réels du restaurant, constituer un petit jeu de validation local, puis comparer objectivement les scores entre le modèle multilingue, un modèle français spécialisé et un modèle arabe.

TABLE 3.3 – Comparaison des approches de sentiment

Approche	Statut dans Tas-tify	Forces	Limites	Pertinence
BERT multilingue NLP Town	Modèle principal	Supporte le français, l'anglais, l'espagnol, l'italien, l'allemand et le néerlandais ; entraîné sur des avis ; utilisable via API [16, 12].	Moins spécialisé qu'un modèle français pur ; dépend d'un service externe si utilisé via API.	Très adapté aux avis courts de restaurant multilingues.
DistilCamemBERT sentiment	Alternative française	Spécialisé français ; modèle plus léger ; résultats documentés sur Amazon Reviews et Allociné [6, 14].	Ne couvre pas naturellement l'arabe ou l'anglais ; nécessite une logique de routage par langue.	Excellent si le périmètre devient majoritairement francophone.
XLM-R / XLM-T	Alternative multilingue	Très bonne base de transfert multilingue ; XLM-R est conçu pour de nombreux langages [5] ; XLM-T est orienté textes sociaux multilingues [2].	Demande un choix de modèle fine-tuné et des validations supplémentaires.	Intéressant pour une version avancée multi-langues.
MARBERT sentiment	Modèle arabe utilisé	Appuyé sur la famille MARBERT, adaptée aux variétés arabes et au texte social [1].	Spécifique aux textes arabes ; ne remplace pas le modèle multilingue pour le français.	Pertinent pour des avis en arabe ou dialecte marocain.
Mots-clés / TF-IDF local	Fallback partiel	Fonctionne hors ligne, transparent et peu coûteux ; le fallback actuel par mots-clés est déjà implémenté.	Moins robuste face à l'ironie, aux négations et au vocabulaire varié ; TF-IDF n'est pas implémenté dans le dépôt.	Utile comme secours, mais insuffisant comme moteur principal.

3.8 Limites de la fonctionnalité

Les limites suivantes sont explicitement identifiées :

- la détection de langue est volontairement simple : elle détecte l’arabe par présence de caractères arabes et classe le reste comme **en** ;
- le fallback local est basé sur des listes de mots positifs et négatifs, pas sur un modèle statistique entraîné ;
- le projet ne fournit pas de métriques d’évaluation sur un corpus d’avis Tastify ;
- la note de 1 à 5 n’est pas combinée avec le commentaire pour améliorer la prédiction ;
- l’utilisation de Hugging Face suppose une configuration correcte du jeton `HUGGINGFACE_API_TOKEN`.

Conclusion

L’analyse de sentiment de Tastify est correctement intégrée à l’architecture applicative : elle est asynchrone, persistée, exploitable dans les tableaux de bord et accompagnée d’un mécanisme de secours. Le modèle principal est adapté au contexte des avis clients, mais une évaluation locale reste nécessaire pour passer d’un prototype académique à une validation de production.

Chapitre 4

Architecture technique et réalisation

Introduction

Ce chapitre décrit la réalisation effective de Tastify. Il présente la stack technique, l'organisation du backend, les deux applications React, les mécanismes de sécurité, le temps réel, les modules métier, les interfaces et le déploiement.

4.1 Stack technique

Le projet adopte une architecture web découplée. Le backend expose des API REST et un canal WebSocket ; les frontends consomment ces services depuis deux applications React.

TABLE 4.1 – Stack technique confirmée

Couche	Technologies	Rôle
Backend	Python, Django, Django REST Framework	API REST, modèles, permissions, logique métier et documentation OpenAPI [9, 11].
Serveur ASGI	Daphne, Django Channels	Exécution asynchrone et WebSocket pour les événements staff [8].
Base de données	MySQL	Persistance relationnelle des données métier [17].
Cache et messages	Redis	Channel layer WebSocket et broker Celery [18].
Tâches asynchrones	Celery, Celery Beat	Analyse de sentiment, traitements de stock, orchestration et tâches planifiées [4].
Frontends	React, Vite, TypeScript, Tailwind CSS	Applications SPA staff et client [15, 20, 19].
Tests	pytest, Vitest, Playwright	Validation backend, unitaire frontend et end-to-end.
Déploiement	Docker Compose	Orchestration des services locaux et préparation au déploiement [10].

4.2 Architecture globale

La figure 4.1 présente l’organisation technique de la solution.

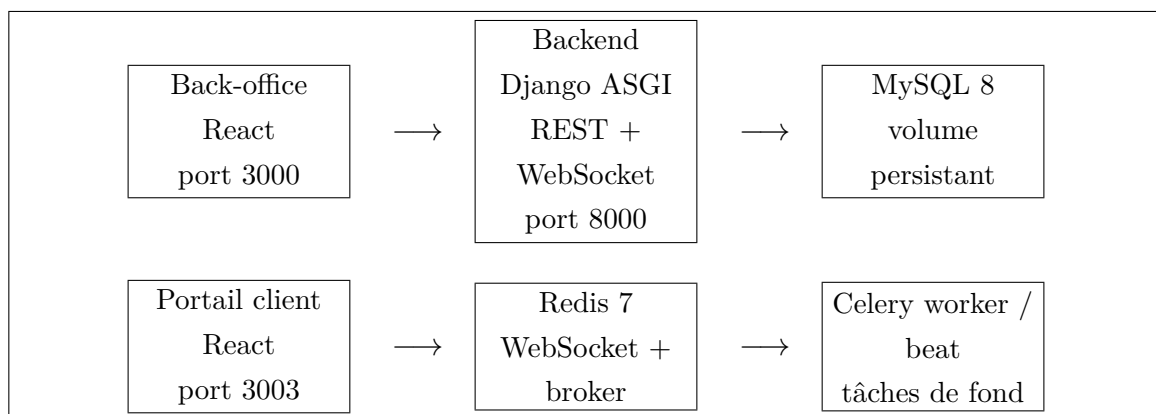


FIGURE 4.1 – Architecture technique de Tastify

Cette architecture a deux avantages principaux : elle sépare les responsabilités et

elle rend le projet déployable par services. Le backend reste le point d'entrée métier ; les frontends se concentrent sur l'expérience utilisateur.

4.3 Organisation du backend

Le backend est organisé en applications Django spécialisées. Ce découpage limite la complexité d'un domaine et facilite les tests.

TABLE 4.2: Modules backend réalisés

Module	Responsabilité
users	Utilisateur personnalisé, rôles, authentification JWT, re-fresh cookies et réinitialisation de mot de passe.
menu	Catégories, plats, disponibilité, images, liens avec les ingrédients et recommandations simples.
tables	Tables, capacités, positions, statuts et textes de plan.
commandes	Commandes, lignes, prix figés, statuts cuisine, signaux et orchestration KDS.
stock	Ingrédients, recettes, mouvements, seuils d'alerte et services d'approvisionnement.
hr	Employés, shifts, offres d'emploi et candidatures.
reservations	Réservations, disponibilité, conflits de créneaux, statuts et notifications.
paiements	Paiements, contributions par ligne, tokens de paiement et réconciliation commande-paiement.
avis	Avis clients, tâche d'analyse de sentiment et stockage du résultat.
analytics	Tableau de bord, revenus, plats populaires, statistiques de sentiment et données météo.
loyalty	Profils, transactions, points et récompenses.
configuration	Identité du restaurant, devise, couleurs et paramètres de service.

4.4 API REST et documentation

Les routes principales sont regroupées dans un routeur DRF. Le dépôt expose notamment les ressources suivantes : catégories, plats, tables, plan-texts, commandes, lignes de

commande, employés, shifts, offres, candidatures, réservations, avis, fidélité, récompenses, paramètres et paiements.

La documentation OpenAPI est générée avec `drf-spectacular`. Les routes `/api/schema/`, `/api/docs/` et `/api/redoc/` permettent de consulter le schéma et de tester l'API pendant le développement.

4.5 Authentification et sécurité

L'authentification repose sur `django-rest-framework-simplejwt`. Le backend émet un access token de courte durée et un refresh token stocké dans un cookie `HttpOnly` [13]. Le dépôt contient également une séparation staff/client afin d'éviter qu'une session d'un portail écrase l'autre.

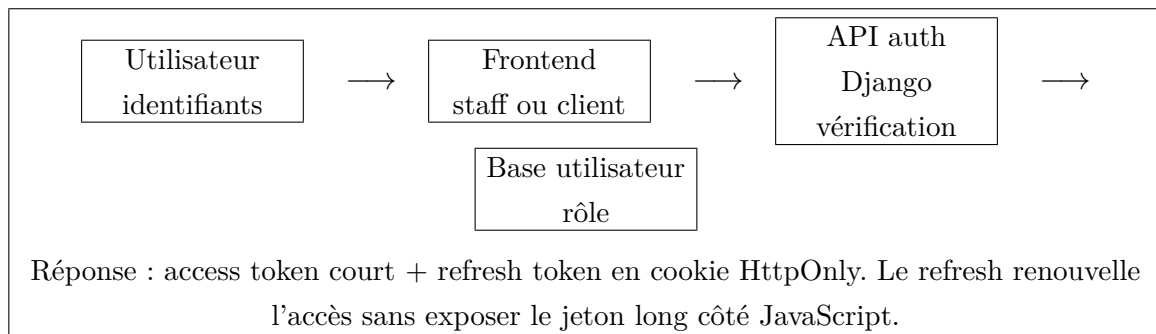


FIGURE 4.2 – Séquence simplifiée d'authentification JWT

Les accès sensibles sont protégés par les permissions DRF et par les gardes de rôles côté frontend. Le gérant accède aux écrans d'administration, le serveur aux fonctions de salle, le cuisinier au KDS, et le client au portail public et à son espace personnel.

4.6 Temps réel et Kitchen Display System

Le temps réel est assuré par Django Channels et Redis. Le canal WebSocket `/ws/staff/` accepte uniquement les rôles internes : gérant, serveur et cuisinier. Les événements staff permettent de mettre à jour les écrans sans rechargement complet.

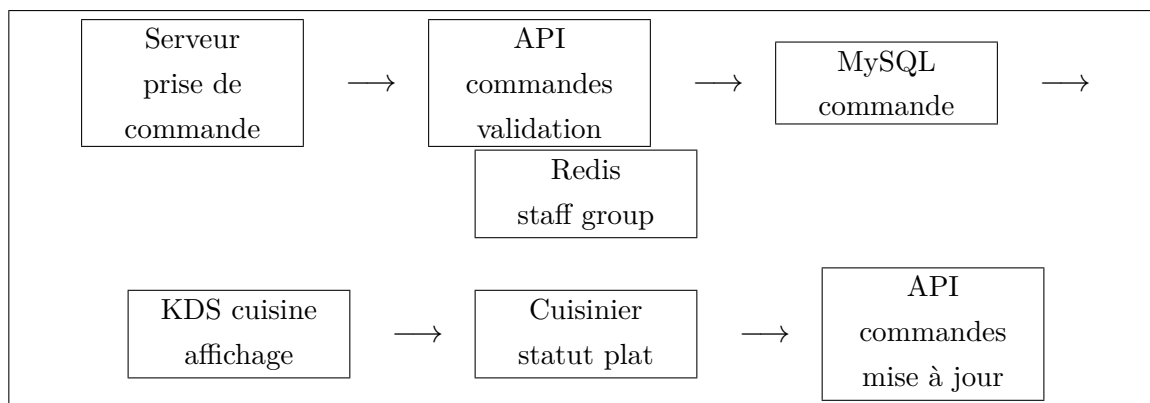


FIGURE 4.3 – Flux temps réel entre prise de commande et KDS

La figure 4.3 montre le flux principal : le serveur crée la commande, le backend la persiste, puis publie un événement vers le groupe staff. Le cuisinier peut ensuite modifier les statuts de préparation.

4.7 Applications frontend

Le dépôt contient deux applications React.

TABLE 4.3 – Applications frontend

Application	Port local	Fonction
Back-office staff	3000	Tableau de bord, menu, catégories, salle, réservations, KDS, stocks, RH, avis et paramètres.
Portail client	3003	Accueil, menu, réservation, compte, inscription, connexion, paiement, contact, fidélité et mode hors ligne.

Les routes frontend confirment la séparation des usages. Dans le back-office, les routes sont protégées par rôle. Dans le portail client, certaines pages sont publiques et d'autres nécessitent une authentification.

4.8 Paiement et fidélité

Le module de paiement associe chaque paiement à une commande et peut relier des contributions aux lignes de commande. Cette structure permet de représenter un

paiement global ou un partage de l'addition. Les méthodes présentes dans le modèle couvrent notamment les espèces, la carte et le QR.

Le module fidélité repose sur un profil lié à l'utilisateur, des transactions de points et des récompenses. Les paliers calculés à partir du solde de points permettent d'ajouter une dimension relation client sans complexifier le parcours principal.

4.9 Stocks et prévision simple

Le module stock relie les plats à leurs ingrédients via une table de recette. Les mouvements de stock permettent de suivre les entrées, sorties et ajustements. Des seuils d'alerte sont présents afin d'identifier les ingrédients à surveiller.

Le dépôt contient également un service d'approvisionnement et des éléments analytics liés à la météo. Aucune preuve d'un modèle prédictif avancé entraîné localement n'a été trouvée ; le rapport présente donc cette partie comme une estimation ou une logique métier simple, et non comme un système d'intelligence artificielle complet.

4.10 Interfaces réalisées

Les captures existantes du dépôt sont intégrées comme preuves visuelles des modules principaux. Elles ne sont pas générées artificiellement.

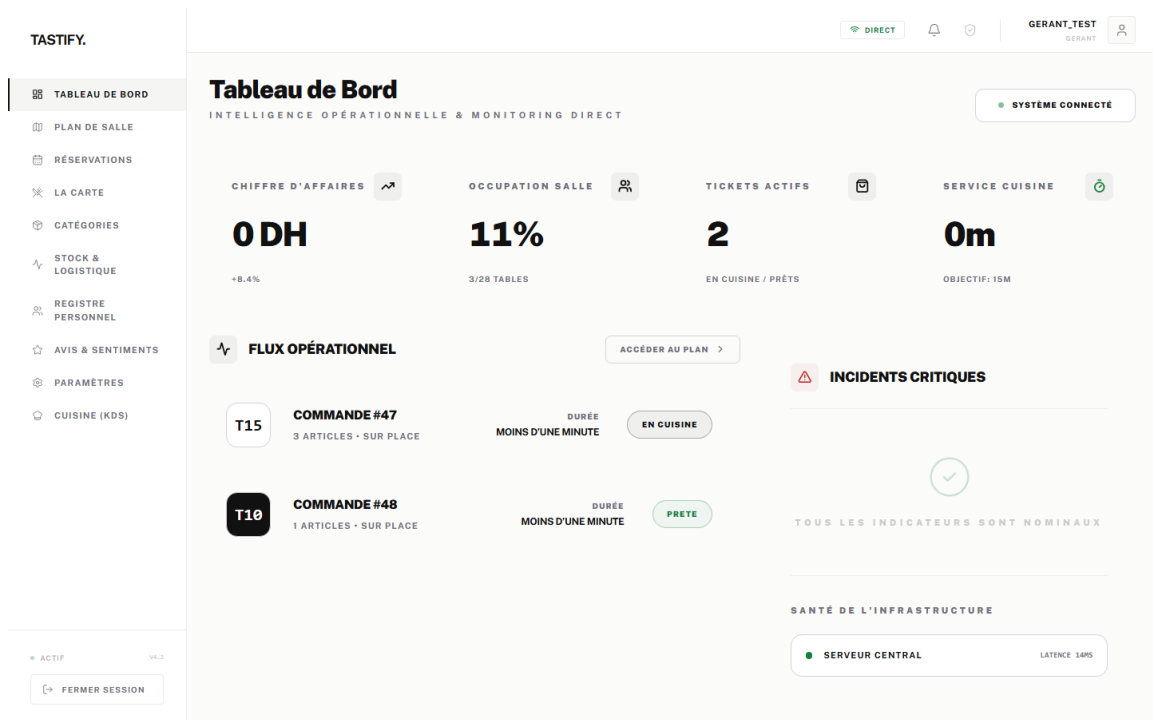


FIGURE 4.4 – Tableau de bord gérant du back-office

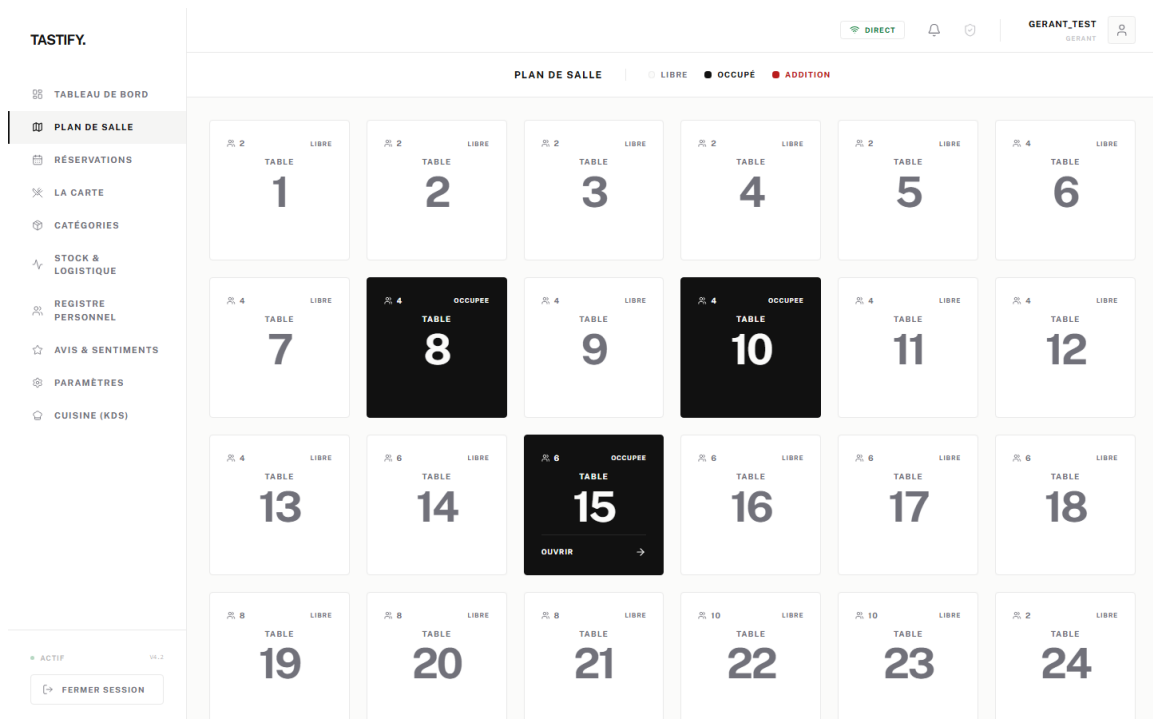


FIGURE 4.5 – Interface serveur : plan de salle

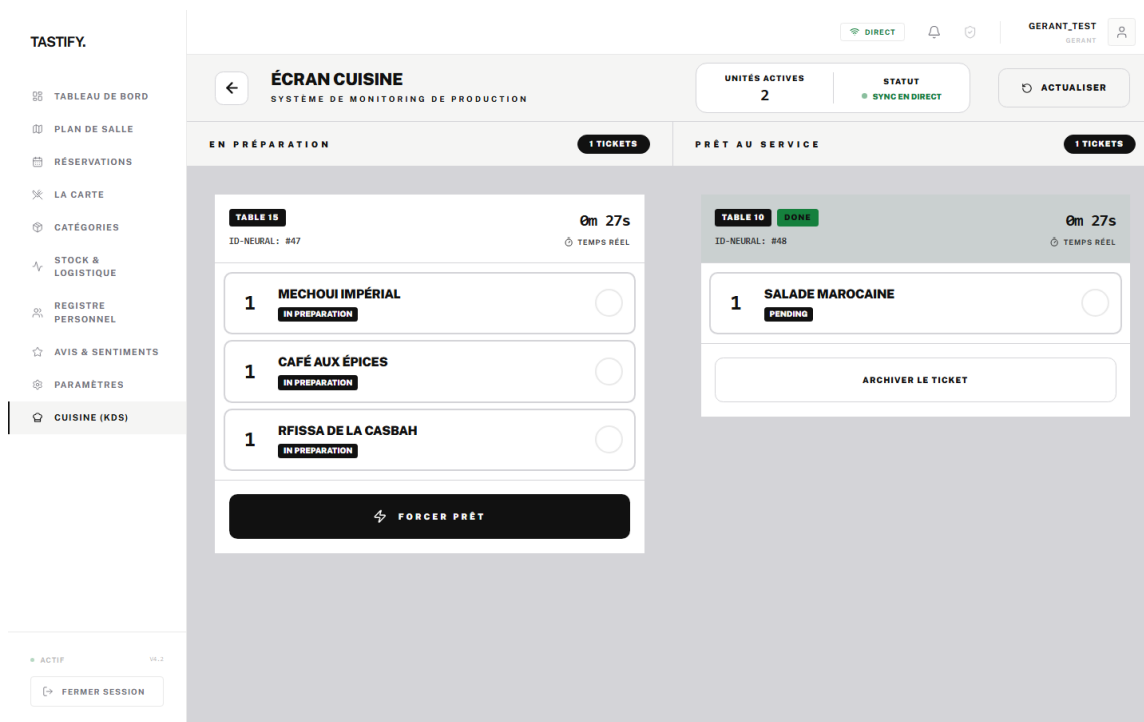


FIGURE 4.6 – Kitchen Display System côté cuisine

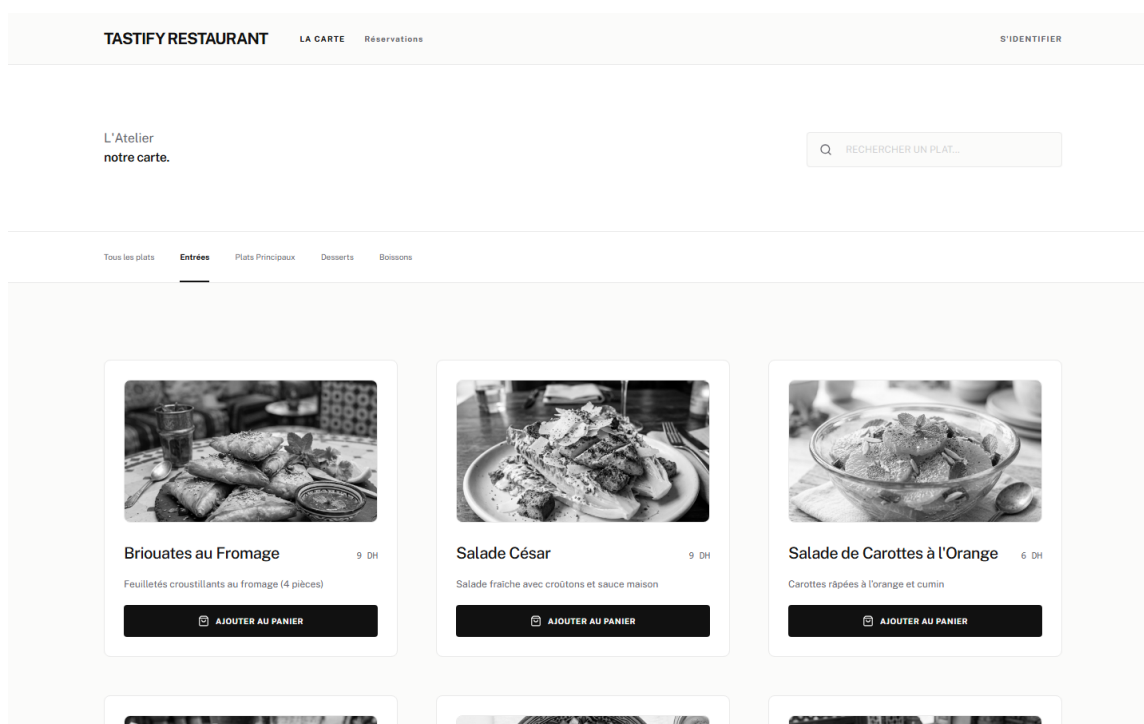


FIGURE 4.7 – Portail client : consultation du menu

Les figures 4.4, 4.5, 4.6 et 4.7 couvrent les quatre espaces essentiels : pilotage, salle, cuisine et client. Les captures complémentaires à fournir sont listées en annexe.

4.11 Déploiement Docker Compose

Le fichier `docker-compose.yml` orchestre les services de développement : base de données, Redis, backend, worker, beat, back-office et portail client.

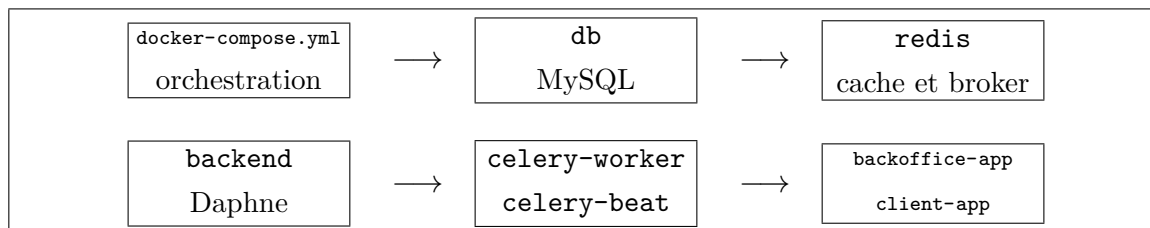


FIGURE 4.8 – Organisation du déploiement Docker Compose

Cette organisation prépare le projet à un déploiement sur serveur, tout en conservant un lancement reproductible en environnement de développement.

Conclusion

La réalisation de Tastify s'appuie sur une architecture moderne et cohérente : backend Django modulaire, API REST, WebSocket, tâches asynchrones, deux frontends React et déploiement Docker Compose. Les choix techniques répondent directement aux besoins métier formulés dans les chapitres précédents.

Chapitre 5

Tests, validation, limites et perspectives

Introduction

La validation d'un projet multi-modules doit couvrir plusieurs niveaux : backend, frontend, parcours utilisateurs, sécurité et intégration. Cette section présente les surfaces de tests observées dans le dépôt, les scénarios importants, les limites vérifiées et les perspectives d'amélioration.

5.1 Stratégie de tests

Le dépôt confirme trois familles principales de tests.

TABLE 5.1 – Familles de tests présentes dans le dépôt

Famille	Outils	Objectif
Backend	pytest, pytest-django	Vérifier les modèles, permissions, API, services, signaux et tâches.
Frontend unitaire	Vitest, Testing Library	Tester des composants, stores et fonctions API côté React.
End-to-end	Playwright	Simuler des parcours utilisateur sur les applications staff et client.
CI et qualité	GitHub Actions, npm scripts	Automatiser lint, typecheck, build, tests et contrôles de sécurité.

Le rapport ne présente pas de résultat chiffré récent d'exécution complète, car aucune

exécution globale n'a été relancée dans le cadre de la rédaction. Les commandes et suites existent, mais les résultats doivent être générés au moment de la soutenance ou de la livraison finale.

5.2 Tests backend

Les tests backend sont répartis dans les applications Django. Ils couvrent notamment :

- l'authentification, les rôles et les permissions ;
- les réservations, conflits de créneaux et services associés ;
- les commandes, lignes de commande, KDS et synchronisation avec les tables ;
- les paiements, tokens, services et signaux ;
- les stocks, mouvements, seuils d'alerte et intégration avec les commandes ;
- les avis et l'analyse de sentiment, y compris le fallback local ;
- les WebSocket staff et l'autorisation des rôles internes ;
- la configuration restaurant et les tableaux de bord.

TABLE 5.2 – Exemples de scénarios backend critiques

Domaine	Scénario à valider
Avis	Un client crée un avis, une tâche Celery est déclenchée et le résultat d'analyse est enregistré.
Sentiment	Hugging Face renvoie un label ; si l'API échoue, le fallback local produit un résultat.
RBAC	Un client ne peut pas accéder aux actions réservées au gérant ou au staff.
KDS	Une ligne de commande passe par les statuts attendus et reste visible côté cuisine.
Stock	Une commande peut déclencher une consommation d'ingrédients selon les recettes.
Paiement	Un paiement valide réconcilie la commande sans montant nul ou négatif.
Réservation	Deux réservations incompatibles sur la même table et le même créneau sont évitées.

5.3 Tests frontend et end-to-end

Les deux frontends possèdent des configurations Vitest et Playwright. Les tests end-to-end couvrent des parcours publics, authentifiés et inter-applications.

TABLE 5.3 – Parcours frontend à valider

Application	Parcours attendus
Back-office	Connexion staff, tableau de bord, menu, catégories, plan de salle, prise de commande, KDS, réservations, stocks, RH, avis, paramètres.
Portail client	Accueil, menu public, inscription, connexion, réservation, compte, panier, paiement, contact et fidélité.
Cross-app	Une réservation ou une commande côté client doit se refléter dans le back-office lorsque le scénario le prévoit.
Accessibilité	Navigation clavier, libellés, états d'erreur, pages not found et mode hors ligne.

5.4 Validation de l'analyse de sentiment

La validation minimale de la fonctionnalité sentiment doit couvrir les scénarios suivants :

1. avis positif non arabe avec réponse Hugging Face de type **5 stars** ;
2. avis négatif non arabe avec réponse **1 star** ;
3. commentaire arabe routé vers `moussaKam/MARBERT-sentiment` ;
4. indisponibilité Hugging Face entraînant l'utilisation de `mots-cles-simple` ;
5. commentaire vide ne créant pas d'analyse ;
6. agrégation correcte des labels dans le tableau de bord.

Ces scénarios sont importants car ils testent à la fois la logique métier, l'appel externe, la normalisation des labels, la persistance et l'exploitation analytique.

5.5 Tests de charge

Le dépôt contient une configuration `load-tester` dans `docker-compose.ci.yml`, qui référence `scripts/locustfile.py`. Cependant, ce fichier n'est pas présent dans l'arborescence analysée. Il serait donc incorrect de présenter des résultats Locust comme réalisés.

La conclusion rigoureuse est la suivante : une campagne de charge est prévue dans l'architecture de test, mais elle doit être complétée par un fichier Locust effectif, puis exécutée pour produire des indicateurs tels que temps moyen, p95, taux d'échec et volume de requêtes.

5.6 Limites du projet

Les limites identifiées sont les suivantes :

- le projet reste un prototype académique et ne fournit pas de données de production ;
- l'analyse de sentiment n'est pas évaluée sur un corpus d'avis Tastify annoté manuellement ;
- le fallback local de sentiment est volontairement simple ;
- la recommandation de plats n'est pas un modèle IA personnalisé complet ;
- la prévision de stock semble reposer sur des règles ou moyennes simples, pas sur un modèle statistique avancé validé ;
- les tests de charge ne sont pas exploitables tant que `scripts/locustfile.py` est absent ;
- certaines informations administratives de la page de garde restent à confirmer.

5.7 Perspectives

Plusieurs améliorations peuvent être envisagées :

- constituer un jeu d'avis annotés pour évaluer objectivement les modèles de sentiment ;
- combiner commentaire et note client afin d'améliorer la fiabilité du sentiment ;
- remplacer ou compléter le fallback par un modèle local léger de type TF-IDF et classifieur linéaire ;
- améliorer la détection de langue, notamment pour le français, l'arabe dialectal et les textes mixtes ;
- ajouter une vraie campagne Locust et intégrer ses métriques dans la CI ;
- enrichir la recommandation de plats avec l'historique client et les contraintes de stock ;
- consolider le déploiement production avec sauvegardes, HTTPS, monitoring et rotation des secrets.

Conclusion

Le dépôt présente une surface de validation sérieuse pour un PFA : tests backend, tests frontend, scénarios Playwright et CI. Les limites sont clairement identifiées et ouvrent des perspectives réalistes. Cette transparence renforce la crédibilité du rapport, car elle distingue ce qui est implémenté, ce qui est prévu et ce qui reste à prouver.

Conclusion générale

Le projet Tastify a permis de concevoir une application web complète pour la gestion d'un restaurant. Le périmètre confirmé par le dépôt couvre les fonctions essentielles : menu, tables, commandes, cuisine, stocks, ressources humaines, réservations, paiements, fidélité, avis clients, tableaux de bord et configuration.

Sur le plan technique, Tastify met en oeuvre une architecture moderne : backend Django REST Framework, base MySQL, Redis, Celery, Django Channels, deux applications React et Docker Compose. Cette architecture répond à un besoin central du domaine : faire circuler l'information rapidement entre les acteurs du restaurant, tout en conservant une séparation claire entre logique métier, interface staff et portail client.

L'analyse de sentiment constitue une valeur ajoutée importante. Elle transforme les commentaires clients en indicateurs simples à suivre par le gérant. Le rapport a volontairement décrit cette fonctionnalité de manière rigoureuse : modèle Hugging Face réellement utilisé, modèle arabe, fallback local par mots-clés, données exploitées, conversion des labels et limites de validation.

Le projet reste perfectible, comme tout prototype académique ambitieux. Les priorités d'amélioration concernent l'évaluation quantitative du sentiment, la consolidation des tests de charge, l'amélioration de la recommandation de plats, l'enrichissement des prévisions de stock et la préparation d'un déploiement de production plus robuste.

En conclusion, Tastify représente une solution cohérente, modulaire et défendable pour un Projet de Fin d'Année. Il démontre la capacité à concevoir un système full-stack réaliste, à articuler plusieurs modules métier, à intégrer du temps réel et de l'asynchronisme, et à documenter les choix techniques avec honnêteté.

Annexe A

Annexes

A.1 Schémas existants intégrés

Le tableau A.1 indique les schémas existants retenus dans le rapport.

TABLE A.1 – Schémas existants intégrés

Fichier	Emplacement	Légende	Objectif
figure_08.jpeg	Chapitre 2	Diagramme de cas d'utilisation du back-office gérant	Montrer le périmètre administratif.
figure_05.png	Chapitre 2	Diagramme de cas d'utilisation des modules salle et cuisine	Expliquer les interactions serveur/cuisine.
figure_02.png	Chapitre 2	Diagramme de cas d'utilisation du portail client	Montrer les fonctions client.
figure_03.png	Chapitre 2	Diagramme de classes de Tastify	Présenter les entités métier majeures.
figure_06.png	Page de garde	Logo Tastify	Identifier visuellement le projet.

A.2 Schémas techniques créés dans le rapport

Les schémas suivants ont été créés directement en LaTeX sous forme de diagrammes textuels. Ils ne sont pas des captures d'écran.

TABLE A.2 – Schémas techniques créés

Schéma	Emplacement	Contenu attendu
Architecture fonctionnelle	Chapitre 2, figure 2.5	Deux frontends, backend DRF, MySQL, Redis, Celery et Hugging Face.
Pipeline de sentiment	Chapitre 3, figure 3.1	Création avis, tâche Celery, API Hugging Face, fallback local, stockage et dashboard.
Architecture technique	Chapitre 4, figure 4.1	Ports, backend ASGI, base de données, Redis, worker et beat.
Séquence JWT	Chapitre 4, figure 4.2	Connexion, vérification, access token, cookie refresh et refresh ultérieur.
Flux KDS	Chapitre 4, figure 4.3	Serveur, API commandes, base, Redis, KDS et cuisinier.
Déploiement Docker Compose	Chapitre 4, figure 4.8	Services Compose et dépendances principales.

A.3 Captures existantes réutilisées

Les captures suivantes sont déjà présentes dans le dépôt et intégrées dans le rapport.

TABLE A.3 – Captures existantes réutilisées

Fichier	Section, légende et objectif
<code>capture_dashboard.png</code>	Section : interfaces réalisées. Légende : tableau de bord gérant du back-office. Objectif : montrer le pilotage et les indicateurs.
<code>capture_salle.png</code>	Section : interfaces réalisées. Légende : interface serveur, plan de salle. Objectif : montrer la gestion opérationnelle des tables.
<code>capture_kds.png</code>	Section : interfaces réalisées. Légende : Kitchen Display System côté cuisine. Objectif : montrer le suivi de préparation.
<code>capture_client.png</code>	Section : interfaces réalisées. Légende : portail client, consultation du menu. Objectif : montrer l'expérience client.

A.4 Captures à fournir avant dépôt final

Les captures ci-dessous ne doivent pas être créées artificiellement. Elles doivent être prises depuis l'application en fonctionnement.

TABLE A.4 – Captures à fournir ou remplacer avant dépôt final

Fichier	Section	Légende	Objectif
capture_login_staff.png	Réalisation, inter-faces	Page de connexion du back-office staff	Prouver la séparation staff et l'accès sécurisé.
capture_stock_admin.png	Réalisation, stocks	Gestion du stock et alertes de seuil	Illustrer le suivi des ingrédients et alertes.
capture_avis_sentiment.png	Analyse de sentiment ou interfaces	Avis clients et statistiques de sentiment	Relier le pipeline de sentiment à l'interface gérant.
capture_reservation_client.png	Réalisation, portail client	Parcours de réservation client	Montrer la réservation côté client.
capture_checkout_payment.png	Réalisation, paiement	Panier et paiement client	Illustrer l'encaissement et le checkout.
capture_loyalty_client.png	Réalisation, fidélité	Espace fidélité client	Montrer les points et récompenses.
capture_hr_admin.png	Réalisation, RH	Module ressources humaines	Montrer employés, shifts ou candidatures.

A.5 Informations non vérifiables ou incertaines

Les éléments suivants doivent être confirmés avant la version officielle :

- nom officiel de l'encadrant ou du tuteur de l'école ;
- intitulé académique exact du thème si l'école impose une formulation ;
- résultats chiffrés des tests si une exécution complète est réalisée ;
- captures finales prises sur l'environnement de démonstration.

A.6 Commandes utiles

Les commandes ci-dessous sont données à titre de rappel pour vérifier le projet et le rapport.

```
cd docs/rapport-pfa-tastify
pdflatex --disable-installer -interaction=nonstopmode main.tex
bibtex main
pdflatex --disable-installer -interaction=nonstopmode main.tex
pdflatex --disable-installer -interaction=nonstopmode main.tex
```

```
docker compose up -d --build
docker compose exec backend python -m pytest
npm run test
```

Bibliographie

- [1] Muhammad Abdul-Mageed, AbdelRahim Elmadany, and El Moatez Billah Nagoudi. Arbert and marbert : Deep bidirectional transformers for arabic. *Proceedings of ACL-IJCNLP*, 2021.
- [2] Francesco Barbieri, Luis Espinosa Anke, and Jose Camacho-Collados. Xlm-t : Multilingual language models in twitter for sentiment analysis and beyond. *Proceedings of LREC*, 2022.
- [3] Grady Booch, James Rumbaugh, and Ivar Jacobson. *The Unified Modeling Language User Guide*. Addison-Wesley, 2005.
- [4] Celery Project. Celery documentation. <https://docs.celeryq.dev/>, 2026. Consulté en juin 2026.
- [5] Alexis Conneau, Kartikay Khandelwal, Naman Goyal, Vishrav Chaudhary, Guillaume Wenzek, Francisco Guzmán, Edouard Grave, Myle Ott, Luke Zettlemoyer, and Veselin Stoyanov. Unsupervised cross-lingual representation learning at scale. *Proceedings of ACL*, 2020.
- [6] Crédit Mutuel Arkéa. Distilcamembert-sentiment. <https://huggingface.co/cmarkea/distilcamembert-base-sentiment>, 2026. Fiche modèle Hugging Face consultée en juin 2026.
- [7] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert : Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of NAACL-HLT*, 2019.
- [8] Django Channels Project. Django channels documentation. <https://channels.readthedocs.io/>, 2026. Consulté en juin 2026.
- [9] Django Software Foundation. Django documentation. <https://docs.djangoproject.com/>, 2026. Consulté en juin 2026.
- [10] Docker Inc. Docker compose documentation. <https://docs.docker.com/compose/>, 2026. Consulté en juin 2026.
- [11] Encode OSS. Django rest framework documentation. <https://www.django-rest-framework.org/>, 2026. Consulté en juin 2026.

- [12] Hugging Face. Inference providers : Text classification. <https://huggingface.co/docs/inference-providers/tasks/text-classification>, 2026. Consulté en juin 2026.
- [13] Jazzband. Simple jwt documentation. <https://django-rest-framework-simplejwt.readthedocs.io/>, 2026. Consulté en juin 2026.
- [14] Louis Martin, Benjamin Muller, Pedro Javier Ortiz Suárez, Yoann Dupont, Laurent Romary, Éric Villemonte de la Clergerie, Djamé Seddah, and Benoît Sagot. Camembert : a tasty french language model. *Proceedings of ACL*, 2020.
- [15] Meta Open Source. React documentation. <https://react.dev/>, 2026. Consulté en juin 2026.
- [16] NLP Town. bert-base-multilingual-uncased-sentiment. <https://huggingface.co/nlptown/bert-base-multilingual-uncased-sentiment>, 2026. Fiche modèle Hugging Face consultée en juin 2026.
- [17] Oracle. Mysql 8.0 reference manual. <https://dev.mysql.com/doc/>, 2026. Consulté en juin 2026.
- [18] Redis. Redis documentation. <https://redis.io/docs/>, 2026. Consulté en juin 2026.
- [19] Tailwind Labs. Tailwind css documentation. <https://tailwindcss.com/docs>, 2026. Consulté en juin 2026.
- [20] Vite. Vite documentation. <https://vite.dev/>, 2026. Consulté en juin 2026.